

1 GlobalExits

1.1 Introduction

You can use the global exits to customize programs without reference to the service type. Using global exits, you can redirect a service to another service, you can manipulate the service parameters of a service and/or change the service result.

1.2 Availability

The GlobalExits are available as of SP8.

1.3 Exits

Via exits, the system provides entry points to enable changes to the defined behavior by programming.

1.3.1 Available user exits

User exits provide entry points that are not modified by releases. In case of releases, the interfaces are respected and preserved by MPDV in a backwards compatible manner.

1.3.1.1 sdiGlobalModifyRequest

Using this exit, you can redirect a service to another service and/or change the service parameters. For example, you might want to redirect a standard service to a custom service without having to change the client.

1.3.1.2 sdiGlobalAddResultTransformationCallbacks

Using this exit, you can register callbacks to influence the result processing.

The system requests the callback "ISdiGlobalResultTransformationFunction" for each data record. After the last data record, the system again requests the callback and uses a dummy data record. Because the system uses a dummy data record, you can add new data records even if the service would otherwise not supply any data records.

1.3.1.3 sdiGlobalCleanup

Using this exit, you can close resources like DB connections or files if you have opened the resources in a previous exit. The system always calls this exit, also in case of an error during service execution.

1.3.2 Specifications for the implementation class

Package name: You must include the class in a package that consists of the domain name (in lower case letters). Further subpackages are **not** allowed.

Example: The service name is "MDUserAccountRules.list", consequently the package is called "mduseraccountrules".

Class name: The class must have a name of the following structure: domain name in lower case letters, where the first letter is capitalized, the name of the service function follows and is written in lower case letters, where the first letter is again capitalized.

Example: The service name is "MDUserAccountRules.list", consequently the class is called "MduseraccountrulesList".



For custom class names, the following definition applies:

Custom names include a "_" (see naming conventions):

Remove the underscore and capitalize the first letter after this underscore.

Example: U_CUST_Units_sample.list => UCustUnitsSampleList

Implemented interfaces / methods: no specifications

Other: The class must have a default constructor without parameters.

Compilation: For the compilation, the Jar files *MpdvDomCoreSdiCompileLib.jar* and *MpdvDomCoreUserExitCompileLib.jar* must be included in the class path.

Deployment: File the class file of the exit in the userexit directory including package directory structure.

The user exit directory is

`jdir/MOC/<InstanceNo>/userexit/<scope>`

or

`jhydradir/MOC/<Man InstanceNo dant>/userexit/<scope>.`

Example: Exit sdiGlobalCleanup for service "MDUserAccountRules.list":

```
package mduseraccountrules;
import de.mpdv.customization.userExit.IUserExitParam;

/**
 * Sample user exit
 *
 */
public class MduseraccountrulesList
{
    public void sdiGlobalCleanup(final IUserExitParam param)
```

```
{
    // TODO implementation
}
```

Directory structure on the server (Instance 1, Scope custom):

`jdir/MOC/1/userexit/custom/mduseraccountrules/MduseraccountrulesList.class`

1.3.3 Interfaces

1.3.3.1 Class: GlobalUeContext

This context class provides data that is globally useful in the context of exits.

Field	Description
hydraNow	The property "hydraNow" is a time stamp created at the beginning of web service processing and, as a result, can be used as reference time stamp for the current web service call.
userId	Includes the user logged on to the client.

1.3.3.2 Exit sdiGlobalModifyRequest

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiGlobalModifyRequestParam	Parameter structure of the exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services.
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiGlobalModifyRequestResult	Result structure of the user exit

1.3.3.3 Class SdiGlobalModifyRequestParam

Field	Type	Description
-------	------	-------------

columnConfigurator	ColumnConfigurator	Includes the columns requested by the client.
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"
filterParametersRootExpression	SqlFilterComplexExpression	Root nodes of the filter parameter in a tree structure
serviceId	String	The service ID of the currently executed service
resultBuilder	ISdiGlobalModifyRequestResultBuilder	Builder to create exit results

1.3.3.4 Interface ISdiGlobalModifyRequestResultBuilder

All method requests are optional, except build(). You may only request the replace...() methods including objects that have actually been changed.

Method	Description
replaceColumnConfigurator(modifiedColumnConfigurator: ColumnConfigurator): ISdiGlobalModifyRequestResultBuilder	Overwrites the column configuration.
replaceSpecialParameters(Map<String, SpecialParam>: modifiedSpecialParameters): ISdiGlobalModifyRequestResultBuilder	Overwrites special parameters.
replaceFilterParametersRootExpression(SqlFilterComplexExpression: modifiedFilterParametersRootExpression): ISdiGlobalModifyRequestResultBuilder	Overwrites filter parameters.
replaceServiceId(String: modifiedServiceId): ISdiGlobalModifyRequestResultBuilder	Overwrites the service ID and thus, turns the service.
build():ISdiGlobalModifyRequestResult	Generates the return structure of the exit.

1.3.3.5 Interface ISdiGlobalModifyRequestResult

You must **not** implement this interface. You must generate the concrete instance using the builder of the field "resultBuilder" of the parameter object.

Method	Description
getColumnConfigurator():ColumnConfigurator	The requested column after the exit processing
getSpecialParameters():Map<String, SpecialParam>	The special paramter after exit requests
getFilterParametersRootExpression():SqlFilterComplexExpression	The root node of the filter parameter in the tree structure after being processed in the exit
getServiceId():String	The service ID (also called function ID or service name) of the service after processing in the exit. You can use this method to redirect a service to another service.

1.3.3.6 Exit sdiGlobalAddResultTransformationCallbacks

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiGlobalAddResultTransformationCallbacksParam	Parameter structure of the exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

Return key in IUserExitParam:

Key name	Type	Description
result	SdiGlobalAddResultTransformationCallbacksResult	Result structure of the

		user exit
--	--	-----------

1.3.3.7 Class

SdiGlobalAddResultTransformationCallbacksParam

Field	Type	Description
columnMap	Map<String, String>	Includes the assignment of acronym => table column (including alias).
specialParameters	Map<String, SpecialParam>	Assigns acronym => SpecialParameter for all web service parameters of the type "is SpecialParameter"

1.3.3.8 Class

SdiGlobalAddResultTransformationCallbacksResult

Field	Type	Description
callbackList	List<ISdiGlobalResultTransformationFunction>	Includes the list of callbacks that the system calls per result row. The list must never be NULL.

1.3.3.9 Interface

ISdiGlobalResultTransformationFunction

Callback function for the result manipulation.

Method	Description
transform (SdiGlobalResultTransformationFunctionParameter	The system calls this method for each result row and permits to change, delete or add rows. The

functionParameter):ISdiDataRowStream	result row is included in the SdiGlobalResultTransformationFunctionParameter. Note: You must create each row of type ISdiDataRow in ISdiDataRowStream using a factory from SdiGlobalResultTransformationFunctionParameter. You must NOT implement the interface ISdiDataRow. You will find an example further down.
--------------------------------------	---

1.3.3.10 Class

SdiGlobalResultTransformationFunctionParameter

Field	Type	Description
dataRowPrototypeFactory	ISdiDataRowPrototypeFactory	Factory method to create an empty data row using the service configuration. You can only create data rows and no rows AdditionalInfoOnly. If you want to include only AdditionalInfo in a row, you must use the factory "ISdiDataRowBuilder" of the method "getDataRowBuilder()".
dataRow	ISdiDataRow	Current result row. Depending on the row type (DataRowType), not all methods are possible. This should absolutely be checked before processing.
getDataRowBuilder()	ISdiDataRowBuilder	Factory method to create a builder. The user can create AdditionalInfoOnly data rows with

		this builder. Note: You require a new ISdiDataRowBuilder for each row. You cannot use the ISdiDataRowBuilder a second time.
--	--	--

1.3.3.11 Enum DataRowType

Type or result data row.

Value	Description
DATA_RECORD	Normal data row
ADDITIONAL_INFO_ONLY	This row includes only AdditionalInfo.
AFTER_LAST_ROW_DUMMY_RECORD	Dummy row that identifies the end of the result rows. All methods except the DataRowType query throw an exception.

1.3.3.12 Exit sdiGlobalCleanup

Parameter key in IUserExitParam:

Key name	Type	Description
param	SdiGlobalCleanupParam	Parameter structure of the exit
context	InterpretedJavaServiceUeContext	Context structure for all exits of the interpreted Java Services.
factory	ISystemUtilFactory	Utility class to access system utilities, such as logger or DB connection in exits.

No result

1.3.3.13 Class SdiGlobalCleanupParam

For future use. The parameter class is empty.

Field	Type	Description
-------	------	-------------

1.4 HowTos

1.4.1 Redirecting a service request to another service

Sample

scenario:

You want to redirect the service MDUnits.list to your custom service U_MDUnits.list.

You use the exit "sdiGlobalModifyRequest" for the service MDUnits.list (original service). If the class MdUnitsList already exists in your scope, only create the method "sdiGlobalModifyRequest" in the existing class. Otherwise, create the class MdunitsList in the package mdunits.

Details on how to create class and package names are included in the section "Specifications for the implementation class".

```
package mdunits;

import de.mpdv.customization.userExit.IUserExitParam;
import de.mpdv.sdi.data.ue.ISdiGlobalModifyRequestResult;
import de.mpdv.sdi.data.ue.SdiGlobalModifyRequestParam;

public class MdunitsList
{
    public void sdiGlobalModifyRequest(final IUserExitParam exitParam)
    {
        final SdiGlobalModifyRequestParam param = exitParam.get("param");
        final ISdiGlobalModifyRequestResult result = param.getResultBuilder()
            .replaceServiceId("U_MDUnits.list")
            .build();
        exitParam.set("result", result);
    }
}
```

Compile this class and store the compiled class including package directory "mdunits" in the userexit folder.

The user exit folder is located at: jdir/MOC/<InstanceNo>/userexit/<scope>

or here: <JHYDRADIR>/MOC/<InstanceNo>/userexit/<scope>.



The system does not request the global exits of the target service, in the example of U_MDUnits.list. Instead, the system requests the global exits of the original service, in the example MDUnits.list.

If you require a modification of the service result, insert the method sdiGlobalModifyRequest in the exit class of the original service (in the example MdunitsList).

If you want to close resources, insert the method sdiGlobalCleanup in the exit class of the original service (in the example MdunitsList).

1.4.2 Creating additional result rows

The following example shows how to insert additional rows at the end of a list service, for example, for totals rows.

You can find general information on creating user exits in the instructions [Java Userexit with IntelliJ IDEA.pdf](#) and [Java Userexit Privacy Hide Columns.pdf](#). The second documentation also contains another example of global user exits.

Here the service BOPerson.list is used. The service lists HR master data. In the example, we insert two additional records at the end, where we set the personnel number to 0 and output a text containing the number of listed persons instead of the name of a person.

Further details follow the source code of the user exit *BopersonList.class*.

```
package boperson;

import de.mpdv.customization.userExit.IUserExitParam;
import de.mpdv.sdi.data.ISdiDataRow;
import de.mpdv.sdi.data.ISdiDataRow.DataRowType;
import de.mpdv.sdi.data.SdiEagerDataRowStream;
import de.mpdv.sdi.data.ue.SdiGlobalAddResultTransformationCallbacksResult;
import de.mpdv.sdi.systemutility.ISdiLogger;
import de.mpdv.sdi.systemutility.ISdiLoggerProvider;
import de.mpdv.sdi.systemutility.ISystemUtilFactory;

import java.util.Calendar;
import java.util.Collections;
import java.util.GregorianCalendar;

public class BopersonList {public class BopersonList {
    private int persCount = 0;

    public void sdiGlobalAddResultTransformationCallbacks(IUserExitParam ueParam) {
        ISystemUtilFactory factory = ueParam.get("factory");
        ISdiLogger logger = factory.<ISdiLoggerProvider>fetchUtil("LoggerProvider").fetchLogger(this.getClass());

        SdiGlobalAddResultTransformationCallbacksResult result = new SdiGlobalAddResultTransformationCallbacksResult(
            Collections.singletonList(functionParameter -> {
                if (functionParameter.getDataRow().getDataRowType() == DataRowType.DATA_RECORD) {
                    this.persCount++;
                }

                if (functionParameter.getDataRow().getDataRowType() == DataRowType.AFTER_LAST_ROW_DUMMY_RECORD) {
                    logger.info("Creating dummy data rows");

                    // Create first additional row
                    ISdiDataRow sumRow1 = functionParameter.getDataRowPrototypeFactory()
                        .createSdiDataRow(" " /* Result set name from ServiceParameter.ResultSet */);
                    sumRow1.setDataTableId(" " /* Result set from DataObject.dataTabLabel */);
                    sumRow1.setCellValue(sumRow1.probeColumnIndex("person.name"), "Dummy row 1, Row count " + this.persCount);
                    sumRow1.setCellValue(sumRow1.probeColumnIndex("person.id"), 99999999);
                    sumRow1.setCellValue(sumRow1.probeColumnIndex("person.valid from"), new GregorianCalendar(1999, Calendar.JANUARY, 1));

                    // Create second additional row
                    ISdiDataRow sumRow2 = functionParameter.getDataRowPrototypeFactory()
                        .createSdiDataRow(" " /* Result set name from ServiceParameter.ResultSet */);
                    sumRow2.setDataTableId(" " /* Result set from DataObject.dataTabLabel */);
                    sumRow2.setCellValue(sumRow1.probeColumnIndex("person.name"), "Dummy row 2, Row count " + this.persCount);
                    sumRow2.setCellValue(sumRow1.probeColumnIndex("person.id"), 99999999);
                    sumRow2.setCellValue(sumRow1.probeColumnIndex("person.valid from"), new GregorianCalendar(1999, Calendar.JANUARY, 1));

                    logger.info("Adding dummy data rows to result");
                    return new SdiEagerDataRowStream(sumRow1, sumRow2);
                } else {
                    return new SdiEagerDataRowStream(functionParameter.getDataRow());
                }
            })
        );
        ueParam.set("result", result);
    }
}
```

Source code explanations

If standard processing of the service is completed, the user exit is called again with a dummy data record.

The dummy data record can be identified by RowType

DataRowType.AFTER_LAST_ROW_DUMMY_RECORD.

Additional data records can only be generated using a *ISdiDataRowBuilder* aus der *dataRowPrototypeFactory* . The method *createSdiDataRow(ResultSet)* generates a data record from the service configuration that contains all columns from the service parameters of a *ResultSet*. The *ResultSet* for which the service parameters are to be used is transferred to the method as a filter. The filter refers to the column "Result Set" of the configuration of the service parameters in the repository.

The newly generated data set is then assigned to a *ResultSet* of the service using the method *setDataTableId(dataTabLabel)*. This *ResultSet* usually refers to the column "Data Tab Label" of the configuration of the data object in the repository.



Please note that the *ResultSet* has a different reference for the methods *createSdiDataRow(ResultSet)* and *setDataTableId(dataTabLabel)*. Depending on the type configuration of the service, the *ResultSets* in the *ServiceParameter* and *Dataobject* can be the same or different. Check the column in the Repository.

The following errors may occur if incorrect information is entered:

- No data record is generated.
- A data record with the incorrect columns is created.
- The data set is not output in the expected *ResultSet*.

If the data record is then generated and assigned to the *ResultSet*, the columns can be filled with values. In the example, the methods *ISdiDataRow.setCellValue(...)* and *ISdiDataRow.probeColumnIndex(...)* are used in combination.



Please bear in mind that the order of the columns in the data record may not be identical to the data records previously generated by standard processing. Always check the index of a column for the newly created records, rather than using an index that you have previously memorized in a variable in the result set when processing an ordinary record.

The newly created records are then output with the auxiliary class *SdiEagerDataRowStream*.